Thrashing Returns: Metastable Failures in Prefix-Cached Agent Serving

Akshat Valse Ali Jannesari
Iowa State University

Draft, 25 October 2026

Abstract

LLM agents send a serving engine multi-turn sessions whose turns are separated by tool calls. Prefix caching keeps a session's KV state between turns, so a follow-up turn that hits the cache prefills only its new tokens. We show that this makes the engine's good, high-hit state metastable. On SGLang 0.5.22 serving Qwen2.5-7B-Instruct on one A100 80GB GPU with a 300,000-token KV pool, the closed-population throughput $\mu(n)$ peaks at 0.233 sessions/s near $n = 20$–24 sessions and falls to an all-miss regime by $n = 48$, while an empty server stayed in the good state in single 3000 s runs at arrival rates up to 2.0 times the all-miss capacity $\lambda_{\text{miss}} = 0.0715$ sessions/s and was unstable at 2.5. In a pre-registered paired test at 1.35 and 1.65 λ_{miss}, a burst of 200 sessions left the server at zero cache hits from 100 s after the burst to the end of the run, 7500 s after it, in all 6 runs; 5 of 6 met the registered trap rule, whose slope test certifies a growing backlog, and in the sixth the backlog stayed near 206 sessions. The same arrivals without the burst stayed good in 6 of 6, and a client-side cap of 20 sessions recovered in 6 of 6. A simulator whose cost model was fitted only to server-log timing predicted these outcomes before they were read. With SGLang's native pool the cliff of $\mu(n)$ moves out roughly threefold and λ_{miss} is unchanged within its standard error. We state a quasi-static model of the window, prove that FCFS-type ordering cannot raise the congested service rate above λ_{miss}, compare in simulation how far published mechanisms raise it, and give a pre-deployment diagnostic: measure $\mu(n)$ for the actual workload and set a session cap from it.

1 Introduction

An LLM agent that uses tools sends its serving engine sessions of many turns. Each turn's prompt is the previous context plus the output of the tool just run, and between turns the session is idle while the tool runs. Prefix caching keeps the key–value (KV) state of the previous turn in GPU memory, so a follow-up turn whose prefix is still cached prefills only its new tokens [31, 64]. In our workload a follow-up turn that hits prefills 551 tokens, while a miss prefills the whole prompt, about 10,500 tokens on average; over a session, prefill work with every follow-up hitting is about 16 times smaller than with every turn missing.

We show that this makes the engine's good state, in which almost every follow-up turn hits, *metastable*. In our measurements, at arrival rates between 1.25 and 2 times the all-miss capacity λ_{miss} (session throughput when every turn misses), an empty server carried the load for 3000 s with 91–93% of turns hitting. A transient burst puts more context into the engine than its KV pool holds. A session waiting for a tool has no running request, so its KV is the first to be evicted to admit waiting work; when it returns it misses, the miss costs an order of magnitude more prefill, the queue grows, the gap between a session's turns grows with it, and more idle KV is evicted before its owner returns. Prefill throughput falls to the all-miss rate, the engine completes sessions more slowly than they arrive, and the backlog does not drain (Fig. 1).

Neither ingredient is new. Throughput that collapses when the working sets of the admitted jobs exceed memory is Denning's thrashing [14, 15], and a bad state that a feedback loop sustains after its trigger is gone is a metastable failure [7]. That KV left idle during tool calls is evicted and costly to recompute motivates Continuum, InferCept, MORI and TokenCake [1, 6, 32, 57], and ThunderAgent and NVIDIA Dynamo already treat agent collapse as working-set thrashing [17, 29]. What we add is the feedback and its consequence in an open system: load lengthens the gap between a session's turns, which raises the miss rate, which raises the load. Under the default FCFS ordering, a server that passes steady-state load tests at a rate inside this window is one sufficiently large burst away from a collapse that, in our model and simulator, it does not leave without shedding load below λ_{miss} or bounding the sessions it admits; resident-first ordering (LPM) and TTL pins raise that threshold (§5.5, §7).

Contributions.

- **A pre-registered real-engine test** on SGLang 0.5.22 serving Qwen2.5-7B-Instruct on one A100 80GB GPU with a 300,000-token KV pool, with three paired arms on common random numbers at two registered rates, 1.35 and 1.65 λ_{miss}. A burst of 200 sessions left the engine at zero hits from 100 s after the burst to the end of every run; 5 of 6 runs met the registered trap rule, and in the sixth the backlog stayed flat near 206 sessions. Without the burst the same arrivals stayed good in 6 of 6; with a client-side cap of 20 sessions the burst drained in 6 of 6. The registered verdict is a pass at both rates (§5.3).
- **Measurements that locate the window**: the closed-population thrashing curve $\mu(n)$ at two pool sizes, λ_{miss},

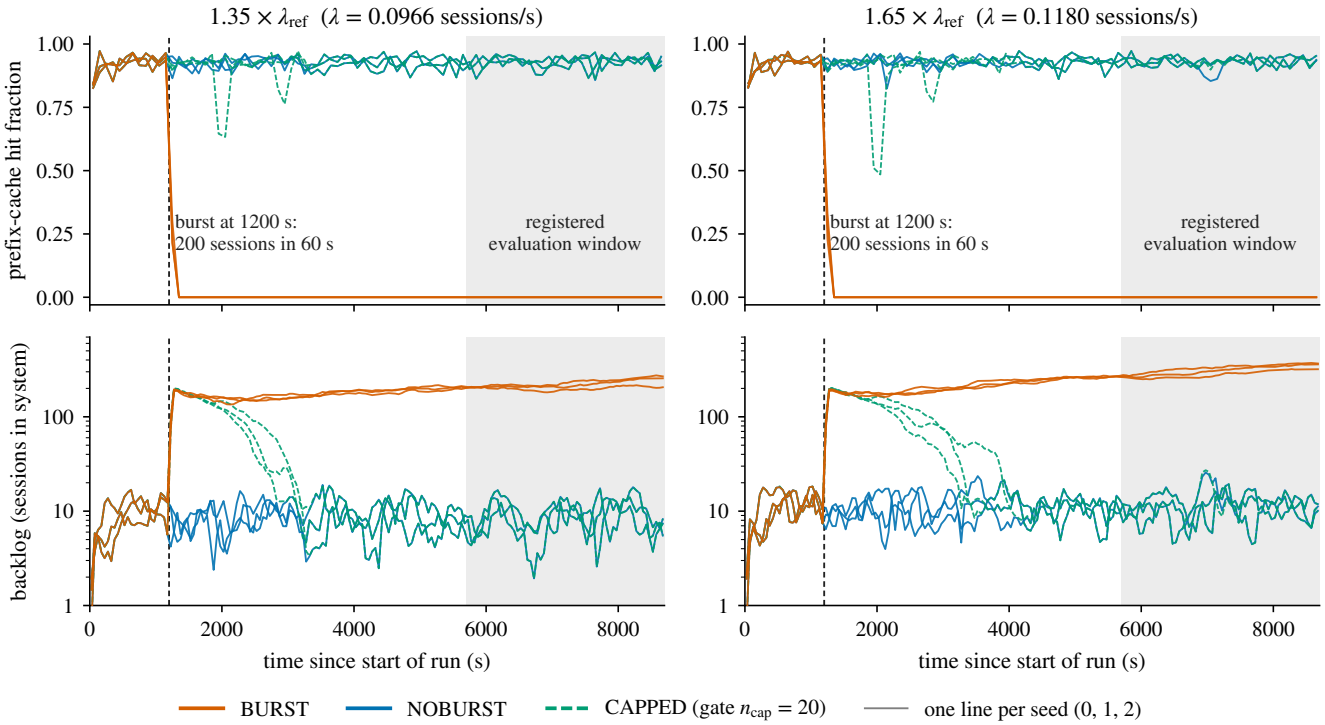


Figure 1: The registered paired test on SGLang (E2) at 1.35 λ_{ref} (left) and 1.65 λ_{ref} (right), λ_{ref} = 0.0715 sessions/s. Top: hit fraction (turns with more than half of the prompt cached) in 100 s windows. Bottom: sessions in the system (log scale), including those in tool calls. One thin line per seed and arm. Dashed: a burst of 200 sessions at 1200 s. Shaded: the registered evaluation window. The hit series is recomputed from the request logs and matches the registered per-run value to four decimals.

and the from-empty stability edge (between 2.0 and 2.5 λ_{miss} in single 3000 s runs). With the pool SGLang sizes itself the cliff moves out roughly threefold and λ_{miss} is unchanged within its standard error, so the cap did not create the cliff (§5.1, §5.2).

- **A quasi-static model and blind predictions.** The window is $(\lambda_{\text{miss}}, \min(\mu^*, \lambda_e))$; under FCFS-type ordering the congested service rate cannot exceed λ_{miss} for any eviction rule, and equals it when the engine stays busy; a session cap N makes the engine a closed N-session system, an elementary observation (§3). A simulator whose cost model was fitted only to server-log timing predicted, before the results were read, the from-empty edge, the E2 outcome and the native-pool curve (§5.4).
- **Published mechanisms and a diagnostic.** In a simulator, with each rule checked against released code where it exists (PEEK; the Continuum preview) and against the paper otherwise (CONCUR), we compare orderings, queue-aware eviction, TTL pins and caps by the congested service rate that sets their recovery threshold; the memory-demand admission that ThunderAgent and NVIDIA Dynamo deploy is not evaluated (§7). We give a pre-deployment procedure (§8); sizing admission from a measured throughput curve is classical load control, and what is specific here is that the from-empty edge bounds the window and that the right cap moves with the tool-time law and the pool.

All real-engine results are for one model, one GPU type and synthetic sessions with exponential 5 s tool times. Results not yet available are marked [pending: . . .]: the native-pool from-empty edge and vLLM's all-miss capacity are running, and the native-pool kill test and the vLLM BURST runs are planned but not submitted (§5.6).

2 Background and what is classical

Engines. LLM engines batch per model iteration: running requests decode one token per iteration and waiting requests join between iterations [61]; KV is stored in pages [31]. SGLang keeps cached prefixes in a radix tree, evicts least-recently-used (LRU) leaves, and can order waiting requests by longest prefix match (LPM) [64]. In SGLang 0.5.22, on our reading of its source (through a page summariser; Appendix D), the default policy is FCFS; LPM falls back to FCFS when more than 128 requests wait; a waiting request does not lock its cached prefix; and under FCFS an LRU refresh periodically touches the prefixes of up to 128 waiting requests. vLLM 0.32.0 schedules FCFS or by priority, and a freed cached block keeps its hash until it is reallocated. Our synthesis of these rules is that in both engines a session's KV is protected only while one of its requests runs.

Thrashing and metastability. Denning showed that admitting one more program to a memory near saturation can collapse efficiency, and that keeping the working sets of the active programs within memory, which limits their number, prevents it [14, 15]. We call throughput as a function of the closed population n the thrashing curve; the term is our framing. LRU does badly on loops larger than the cache [11, 52]. A metastable failure is a bad state that persists after its trigger is removed, sustained by a feedback loop under uncontrolled load; one of Bronson et al.'s case studies is a lost look-aside cache [7], and Huang et al. survey such failures in production systems [25]. Gibbens, Hunt and Kelly showed hysteresis in loss networks with random alternative routing and that trunk reservation, a form of admission control, removes it [21].

What is classical, and what is specific to agent serving. The collapse past the working-set level, the remedy of limiting active jobs, and a bad state that outlives its trigger are classical; we claim no new queueing theory. The fluid picture of §3.3, a unimodal throughput curve that gives one stable and one unstable equilibrium under open arrivals, is the classical drift analysis of bistable random-access channels [19, 30]; network congestion collapse, a stable bad state sustained by retransmissions, is its analogue [26, 40]. Replacing an open system's departure rate by its closed-system throughput is the classical decomposition (flow-equivalent) approximation, whose use to find unstable regimes of thrashing multiprogramming systems goes back to Courtois [12], and choosing the number of admitted jobs from the throughput curve is multiprogramming-level load control [16, 22, 50]. Two features of agent serving make these apply. The reusable KV is exactly the idle KV: a session in a tool call holds no running request, so memory-gated admission reclaims its cache first. And reuse makes a hit much cheaper than a miss, so losing the cache multiplies the work of every later turn. Both features are the premise of earlier agent-serving systems [1, 6, 32, 57], and ThunderAgent and NVIDIA Dynamo describe agent collapse as working-set thrashing [17, 29]. What we add is the loop they close (misses lengthen the queue, the queue lengthens the gap between a session's turns, and longer gaps cause more misses) and its open-system consequence. We measure that loop on a real engine, locate its window, and evaluate published mechanisms in these terms.

3 Model

3.1 Workload and notation

A session has $T \sim \min(\text{Geometric}(1/15), 60)$ turns, so $E[T] = 14.761$. Turn k sends a prompt of $L_k = L_0 + (k-1)(\Delta + d)$ tokens: a first prompt of $L_0 = 2000$ tokens, $\Delta = 550$ tokens of tool output and $d = 100$ decoded tokens per turn. Between turns the session waits for a tool for a time Z, independent across turns, exponential with mean 5 s in the workload of record. A turn *hits* if more than half of its prompt is served from cache; on the real engine a hit reuses everything but the 550 new tokens and the last decoded token, so it prefills 551 tokens. First turns always miss, so the hit fraction is at most $1 - 1/E[T] \approx 0.93$. With every turn missing, a session's prefill work is $W(T) = 2000\,T + 325\,T(T-1)$ tokens with $E[W] = 154{,}529$; with every follow-up hitting it is $2000 + 551(T-1)$, about 9,600 in expectation.

The engine has a KV pool of M tokens. In the *closed* system n sessions are present and a finished session is replaced at once. The thrashing curve $\mu(n)$ is the long-run session completion rate (sessions/s); $\mu^* = \max_n \mu(n)$ is attained at n^*, and $\mu_\infty = \lim_{n\to\infty} \mu(n)$ is the *congested service rate*. The all-miss capacity λ_{miss} is the throughput of the saturated engine with prefix caching disabled. In the *open* system sessions arrive as a Poisson process of rate λ, and $N(t)$ counts sessions in the system (waiting, running or in a tool call). A session rate is estimated either as sessions completed per second or, with much lower variance because session work is heavy-tailed in T, as turns per second divided by $E[T]$ ("turns-based").

3.2 The thrashing curve for agent sessions

Let E_{lb} be the time-averaged KV footprint of a session over its lifetime, a length-biased mean in which long sessions and late turns weigh more (during the tool call after turn k a session holds $L_k + d$ tokens). By the $H = \lambda G$ relation [8, 23], the mean KV demand of n present sessions is nE_{lb}. For our workload E_{lb} is about 10.6k tokens, so a 300k pool holds $N_{\text{lb}} = M/E_{\text{lb}} \approx 28.4$ sessions on average. While nE_{lb} is well below M, idle KV survives the tool call and $\mu(n)$ rises with n through batching. Beyond it, admitting a waiting turn evicts idle KV, which belongs to sessions in tool calls; they miss when they return. The peak n^* lies below N_{lb} because resident demand fluctuates around its mean.

3.3 The open system as a quasi-static approximation

Replacing the departure rate of the open system by its closed-system value at the current population, the classical decomposition approximation (§2), gives

$$\dot{n} = \lambda - \mu(n). \tag{1}$$

Let μ rise to μ^* and fall to $\mu_\infty < \mu^*$. If $\lambda < \mu_\infty$ (and $\mu > \lambda$ above the lower crossing), Eq. (1) has one attracting fixed point. If $\mu_\infty < \lambda < \mu^*$, it has a stable fixed point $n_1 < n^*$ and an unstable one $n_2 > n^*$, and from any $n > n_2$, $n \to \infty$: the good state is metastable. If $\lambda > \mu^*$ there is no good state. From a congested state x_0 the system recovers if and only if $\lambda < \min_{[n_1, x_0]} \mu$, which tends to μ_∞ for deep congestion; μ_∞ is the *recovery threshold*. After a finite burst x_0 is finite, so recovery above μ_∞ is possible; the threshold for a given burst depends on its size.

In a finite system fluctuations can carry N past n_2 without a burst, and an experiment sees only escapes within its horizon. We therefore use a measured *from-empty edge* λ_{e}, the largest rate at which runs started from an empty engine stay stable

over a stated horizon, and define the window

$$\mathcal{W} = \big(\mu_\infty,\ \min(\mu^*, \lambda_e)\big). \tag{2}$$

Inside $\mathcal{W}$ the good state exists from empty, and a burst that crosses the basin boundary leaves the system in a regime whose service rate is below the arrival rate. That regime is transient (the backlog grows), not a second stationary state, so we say metastable rather than bistable.

Eq. (1) assumes that the internal state (turn-index mix, cache contents, queue order) relaxes quickly compared with changes in n. A math check (Appendix D) found this reasonable at fixed points and in deep congestion, where μ is flat, but not near the cliff. We therefore use it for the window and the recovery threshold, not for burst sizes: in an earlier simulator (§4.4) the fluid critical burst $n_2 - n_1$ at $2\lambda_{\text{miss}}$ was 20.5 sessions against an empirical 40, because burst sessions are fresh and small.

3.4 Two results

Proposition 1 (FCFS-type orders, any eviction rule)**.** *Assume (A1) every resident context has at least $L_{\min} > 0$ tokens; (A2) every follow-up request waits in the queue at least w_n, with $w_n \to \infty$ in probability as $n \to \infty$; (A3) a miss in congestion costs the same as in the cache-disabled reference. Then the rate of hit turns is at most $M/(L_{\min} w_n) \to 0$, and $\limsup_{n\to\infty} \mu(n) \le \lambda_{\text{miss}}$. If also (A4) admission is work-conserving and memory-bound, so that the congested engine stays saturated, then $\mu(n) \to \lambda_{\text{miss}}$.*

Proof. A hit at turn $t+1$ needs at least $L_{\min}$ tokens of the session's KV to stay resident from the end of turn t until turn $t+1$ is admitted, at least w_n later. Resident token-time accrues at rate at most M, so (hit rate)$\cdot L_{\min} w_n \le M$. With a vanishing hit fraction, (A3) makes almost every turn cost what it costs in the cache-disabled reference, whose saturated throughput is λ_{miss}, which bounds μ in the limit; under (A4) the engine is saturated and the bound is attained. □

(A2) holds for FCFS and program-level FCFS: turn throughput is bounded, so the queue grows without bound in n (by Little's law the mean cycle time is n over the turn rate), and under FCFS a returning request waits behind essentially every request queued before it. The bound holds for every eviction rule, even Belady's [5], and for protection of queued prefixes that keeps FCFS order, where (A4) can fail: in our earlier simulator a hard pin of queued prefixes under FCFS order gives $0.919\,\lambda_{\text{miss}}$ (§7). A resident-first order such as LPM violates (A2): a returning session whose KV is still resident does not wait.

Remark (resident-first orders; a conjecture). Under LPM in congestion we expect the follow-up hit fraction to tend to $P(Z < R_\infty)$, with R_∞ the congested lifetime of idle KV, so μ_∞^{LPM} exceeds λ_{miss} through tool calls shorter than R_∞; the simulator agrees (§7), but we have no proof.

Proposition 2 (cap identity; elementary)**.** *Let a FIFO gate admit at most N sessions, each holding its slot from admission to the end of its last turn, tool calls included. In the closed system with $n \ge N$ sessions the admitted sessions evolve exactly as a closed system of N, so $\mu_{\text{cap}}(n) = \mu(\min(n, N))$ in law.*

Proof. When an admitted session finishes, the gate admits its head at once and the replacement joins the gate. □

With $N \le n^*$, μ_{cap} is non-decreasing, so under Eq. (1) the capped system has one attracting state if and only if $\lambda < \mu(N)$, and drains a burst of b sessions in about $b/(\mu(N) - \lambda)$: 1571 s at 1.35 and 1889 s at $1.65\,\lambda_{\text{ref}}$ for the registered test (measured against the engine in §5.3). This is the standard closed-system equivalence of a system with a bounded multiprogramming level (§2). The stochastic counterpart (positive recurrence for $\lambda < \mu(N)$) should follow from standard arguments; we have not written them out. A cap $N > n^*$ raises the congested level to $\mu(N)$ but leaves the window $(\mu(N), \mu^*)$ open.

4 Methodology

4.1 Real-engine harness

All real-engine runs used one NVIDIA A100 80GB PCIe GPU per job on the Nova SLURM cluster at Iowa State University, with SGLang 0.5.22 (FlashInfer attention, chunked prefill of 8192 tokens, at most 256 running requests, LRU eviction) serving Qwen2.5-7B-Instruct [48]. The KV pool was capped at 300,000 tokens or, in the native-pool runs, sized by SGLang at 955,757 tokens. Decoding was greedy, 100 tokens per turn (Appendix A lists every setting).

An open-loop client on the same node sends each turn as a completion request whose prompt is a list of token ids: random content from a seeded generator, extended each turn by the token ids the server returned plus 550 new tool tokens, so a hit covers exactly the previous prompt and output; during a tool call the client sleeps. Cached tokens come from the server's usage report, which the engine's counters and log confirmed in Stage C. Before every run the cache is flushed and checked with a canary (a 1024-token request that reused 1023 tokens before the flush must reuse at most 15 after; it reused none). Each arm of the registered test ran in its own job on a fresh server.

The BURST arm adds 200 fresh sessions within 60 s at $t_b = 1200$ s to the Poisson stream; NOBURST is the same stream without them; CAPPED is BURST plus a client-side FIFO gate admitting at most n_{cap} sessions, each holding its slot until its last turn ends (the gate of Proposition 2). The arms of one seed share arrival times, turn counts and tool times (common random numbers), so BURST differs from NOBURST by the burst and CAPPED from BURST by the gate, up to GPU timing nondeterminism: each arm ran on a fresh server in its own job, and the arms of one seed did not always share a node.

4.2 Staged protocol

Each stage's inputs came from the previous one (job IDs in Appendix A). *Stage C* measured $\mu(n)$ with all n sessions starting at $t = 0$, one run per n, a 900 s warmup and an 1800 s window. *E0a* measured λ_{miss} with the radix cache disabled; the reference rate λ_{ref} is its turns-based value, 0.0715 sessions/s, and arrival rates are multiples of it. *E0b* ran Poisson arrivals from an empty engine for 3000 s per rate; a run is stable if its final queue is below 50 and its queue rises by at most 10 over the last 1500 s. The registration was then filled and committed, and *E2* ran the paired arms. Afterwards we measured the curve and λ_{miss} with the native pool, ran an exploratory policy grid, and installed and smoke-tested vLLM.

4.3 The registered kill test

Each E2 run lasts 8700 s and is judged over $[t_b{+}4500,\ 8700]$ s. A BURST run is TRAPPED if (a) its uncached prefill token rate is within ±15% of E0a's all-miss rate, 11,243 tokens/s, and (b) the slope of its backlog (sessions in the system) exceeds $0.5(\lambda - \lambda_{\text{miss}})$. A run is RECOVERED if over the last 300 s its hit fraction exceeds 0.8 and its outstanding backlog (requests in flight plus sessions held at the gate) is below 20; NOBURST is good if RECOVERED. A rate PASSES if, in at least two of three seeds, BURST is trapped, NOBURST good and CAPPED recovered in that same seed. Criterion (a) uses tokens because a trapped engine with a growing backlog completes young, cheap sessions faster than λ_{miss}: on traps simulated in our earlier simulator (§4.4), a session-rate test fired on at most 1 of 29 at 2 λ_{miss} and none of 39 at 2.5, while the token test fired on all 84 traps at 1.5–2.5 λ_{miss}.

The registration was filled by formula from committed copies of the Stage C curve, the E0a calibration and the E0b bracket. The window was $(1,\ 2.0)\,\lambda_{\text{ref}}$, its upper edge E0b's largest all-stable multiple (below $\mu^*/\lambda_{\text{ref}} = 3.26$). The rates followed a rule committed to the runbook before E0b was submitted (`288b801`, 2026-10-22), one third and two thirds of the way across the window rounded to 0.05: 1.35 and 1.65 λ_{ref} (0.0966 and 0.1180 sessions/s). The cap was the smallest n with $\mu(n) \geq 0.95\mu^*$: $n_{\text{cap}} = 20$, since $\mu(20) = 0.224$ is within 4% of $\mu(24) = 0.233$. The file was committed (`d279b7b`) before the first E2 job. The job script refuses to start unless the file is committed and unmodified and the job's cell, rates, cap, reference rates, hardware and protocol match it, and writes its SHA-256 into every run; the analysis calls a verdict decisive only for the registered cell and rates with every run bound to the file and started after its commit (Appendix B).

4.4 The calibrated simulator

Our discrete-event simulator models a continuous-batching engine: iterations of at most 8192 prefill tokens and 256 sequences; admission reserves a request's end-of-turn KV; idle KV of sessions in a tool call or waiting is evicted whole, least recently used first; waiting requests pin nothing. An iteration costs a fixed time plus a per-token term plus attention terms. The five cost parameters were fitted only to SGLang's per-batch timing lines in the Stage 0 and Stage C server logs (decode $R^2 = 0.992$, median error 0.75%; prefill $R^2 = 0.998$, 1.3%), never to $\mu(n)$. It runs mixed prefill–decode iterations or, like SGLang, separate iterations with prefill priority; the primary (mixed) mode was the one closer to the real Stage C $\mu(n)$ under a pre-declared metric (mean relative error 0.104 against 0.127), a margin carried by one cell, and both modes' predictions are reported. It does not model partial prefixes, the waiting-prefix refresh or the overlap scheduler; reserving whole turns at admission is an upper bound on what paged engines hold (in round 5 of the earlier simulator, on-demand allocation with preemption and SGLang-like ratio reservation kept the curve and the trap).

Validation. The declared level test, simulated μ within 15% of the real value in each of 8 Stage C cells, failed in both modes: mixed batching missed one cell, FCFS $n = 48$, by +42% (0.070 against 0.049 sessions/s), separate batching two. The shape was reproduced: the hit fraction crosses 0.5 at $n = 32.5$ against 32.2, the plateau ($n = 16$–24) agrees within 9.2% and $n = 300$ within 2%, and $\mu(300)/\mu^*$ is 0.50 against 0.46. The single real run at $n = 48$ dipped below the simulator's all-miss level; its turns-based estimate, 0.063 sessions/s, is within 15%.

Blind predictions. The calibrated simulator then predicted, in both modes (mixed primary), B1: λ_{miss} and the all-miss token rate; B2: the fraction of from-empty runs that stay stable at each rate; B3: the E2 outcome at each rate from 0.09 to 0.16 sessions/s; B4: the closed curve with 600k- and 900k-token pools. They were committed (`64bc4e3`), tagged and pushed before any E0b or E2 result was read and before any native-pool run. The agents that made them were instructed not to contact the cluster and not to open the E0a or E0b result directories or the main checkout; their only real data were the Stage 0 and Stage C runs on which the simulator was calibrated and validated. They were not technically barred: we checked their session records afterwards and found no cluster command and no access to E0b before the tag was pushed. B1 is not blind in time (E0a had been committed), and it is close to the calibration data: the cost model was fitted to Stage C per-batch prefill timing, and the all-miss capacity is essentially a prefill-token rate, so B1 checks the simulator's bookkeeping more than its predictive power. An independent verifier reran a subset, whose outputs matched exactly, and recomputed every B2 and B3 classification (Appendix C).

Simulator rounds 1–5, with assumed costs (all-miss capacity 0.0474 sessions/s), are used for the mechanism comparison of §7, in units of their own λ_{miss}, and in §3.3, §4.3 and §4.4 where stated.

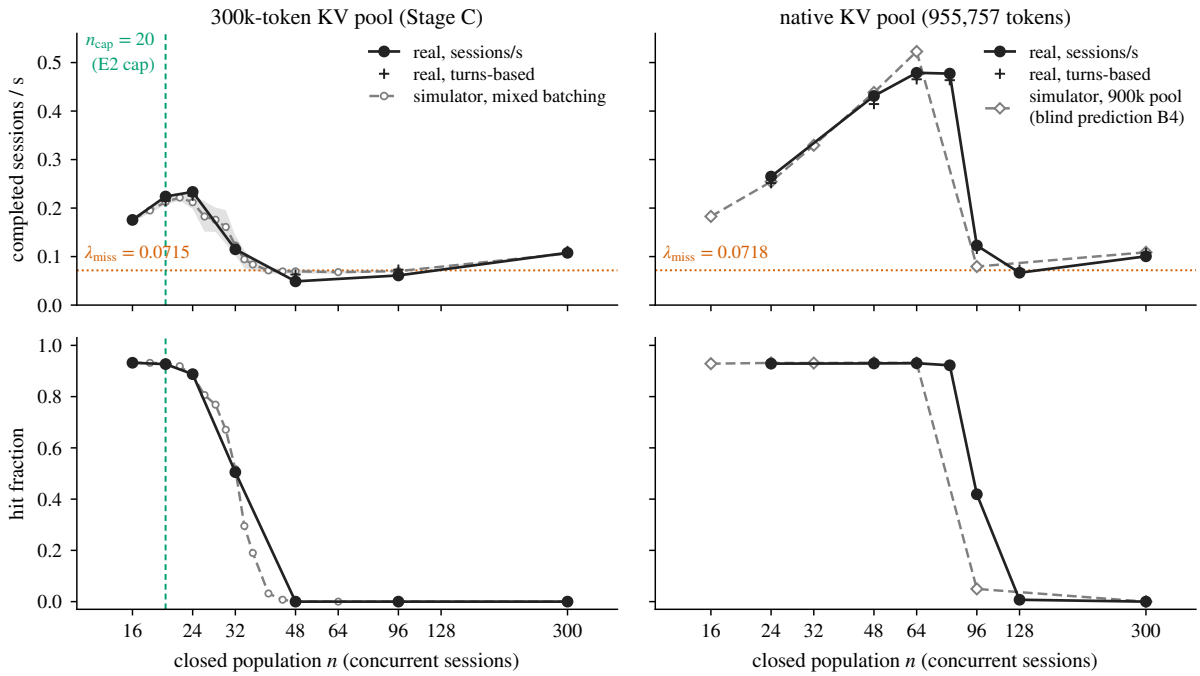


Figure 2: Closed-population thrashing curve, real engine (black, filled; one run per n) against the calibrated simulator (gray, open, dashed). Top: $\mu(n)$ in completed sessions/s (plus signs: turns-based estimate); bottom: hit fraction. Left: 300k-token pool, simulator mean of 3 seeds ±1 s.e. Right: native pool (955,757 tokens), simulator = blind prediction B4 at 900k tokens (seed 0). Dotted: λ_{miss}. Green dashed: the registered cap $n = 20$.

5 Results on the real engine

5.1 The thrashing curve, with a capped and a native pool

With the 300k pool (Fig. 2, left), $\mu(n)$ rises from 0.176 sessions/s at $n = 16$ to 0.224 at 20 and 0.233 at 24, with hit fractions 0.93, 0.93 and 0.89. The run at $n = 32$ mixes two regimes: degraded from about 450 to 2400 s, then recovered to 0.80, for a hit fraction of 0.51. At $n = 48$ none of the 20.0M prompt tokens in the window was served from cache: evictable tokens survive about 6.3 s before eviction (estimated as mean evictable KV over the eviction rate) while a session returns after 30.6 s on average (2.9 s against 66 s at $n = 96$). The cliff is where memory predicts ($N_{\text{lb}} = 28.4$; the hit fraction crosses 0.5 at $n = 32.2$). The rising tail, $\mu(48) = 0.049 < \mu(96) = 0.061 < \mu(300) = 0.108$ sessions/s, is a transient of the synchronized start, whose young sessions have short prompts (71% of the stationary length in flight at $n = 300$); the all-miss level implied by token throughput is roughly flat at 0.072–0.081 sessions/s, so the collapse from the peak is about threefold. Stock LPM keeps 39% of turns hitting at $n = 48$ (rising from 0.28 to 0.44 through the window); at $n = 300$ its queue of 269 exceeds the 128-request threshold at which, by the source, SGLang falls back to FCFS, and no turn hits; whether the fallback causes the zero was not tested (at $n = 300$ the pool may be oversubscribed under any ordering).

With the native pool (Fig. 2, right), SGLang sized the KV pool at 955,757 tokens, 3.19 times the cap. μ peaks at 0.479 sessions/s at $n = 64$ (0.477 at 80) with 92–93% of turns hitting, then falls to 0.123 at $n = 96$ (hit fraction 0.42), 0.067 at 128 (0.007) and 0.101 at 300. The hit fraction crosses 0.5 between $n = 80$ (0.92) and 96 (0.42), near 93 by linear interpolation, against just above 32 at 300k; the peak is at 64–80 against 20–24. With one run per n, the cliff moves out roughly threefold. The all-miss capacity is unchanged within its standard error: 0.0718 ± 0.0037 sessions/s turns-based against 0.0715 ± 0.0038, and 11,282 ± 51 against 11,243 ± 84 all-miss tokens/s. The 300k cap did not create the cliff; it moved it. The ratio $\mu^*/\lambda_{\text{miss}}$ is 6.7 with the native pool and 3.26 at 300k.

5.2 The all-miss capacity and the from-empty edge

E0a gives $\lambda_{\text{miss}} = 0.0715 \pm 0.0038$ sessions/s turns-based (standard error over three seeds), 0.0717 ± 0.0076 in completed sessions, and 11,243 ± 84 all-miss prefill tokens/s. The token rate varies by ±1.2% across seeds and the session rate by ±17%: the engine is bound by prefill tokens, and the sessions completed in 1800 s are a small sample of a heavy-tailed quantity. The token rate divided by $E[W]$ gives 0.0728 sessions/s, within 2% of the turns-based value.

From an empty cache and queue (Fig. 3) the engine stayed stable at 1.25, 1.5, 1.75 and 2.0 λ_{ref}, ending each run with an empty queue and with 91–93% of turns hitting over the run, and was unstable at 2.5 λ_{ref} (final queue 127, rising by 146 over

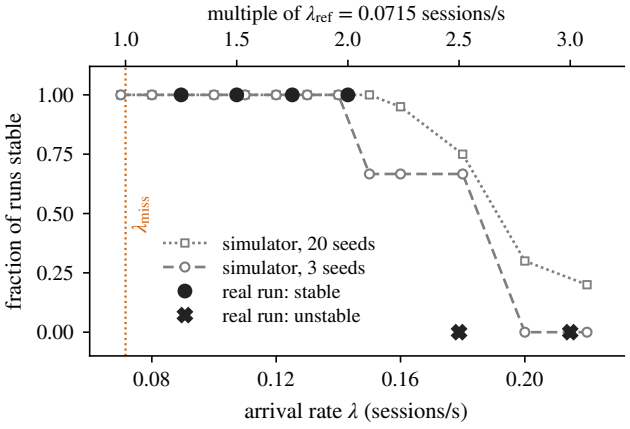


Figure 3: From-empty stability, 300k pool. Real engine (E0b): one 3000 s run per rate (1 = stable, 0 = unstable). Gray: blind prediction B2, fraction of simulated runs stable (3 seeds dashed, 20 dotted). Top axis: multiples of λ_{ref} (the simulator's own λ_{miss} is 0.0722).

Table 1: The registered test (E2). States for seeds 0, 1, 2: T trapped, I indeterminate, G good, R recovered. Hit fraction, uncached prefill rate in multiples of the all-miss rate, mean backlog (sessions in the system) and its slope (sessions/s) over the evaluation window; ranges over seeds. Slope thresholds: 0.0125 at 1.35 and 0.0232 at 1.65 λ_{ref}.

λ/λ_{ref}	arm	states	hit	tokens	backlog	slope
1.35	BURST	T I T	0.000	1.009–1.025	200–232	0.0036–0.0222
	NOBURST	G G G	0.930–0.934	0.08–0.09	7.9–9.7	<0.001
	CAPPED	R R R	0.930–0.934	0.08–0.09	7.9–9.7	<0.001
1.65	BURST	T T T	0.000	1.025–1.029	283–332	0.0277–0.0362
	NOBURST	G G G	0.921–0.934	0.10–0.13	10.9–11.8	<0.002
	CAPPED	R R R	0.929–0.934	0.10–0.11	10.9–11.7	<0.002

the last 1500 s) and 3.0 (289 and 196). The edge lies between 2.0 and 2.5 λ_{ref}, well below $\mu^*/\lambda_{ref} = 3.26$, so the from-empty edge, not the peak, bounds the window. The registered window was (1, 2.0) λ_{ref} = (0.0715, 0.143) sessions/s. Each rate is one 3000 s run; a longer horizon could find a lower edge, and at 2.0 λ_{ref} the hit fraction fell from 0.934 in [300, 1500) s to 0.903 in the second half of the run.

5.3 The registered paired test

The registered analysis, run on all 18 runs with the code frozen at the registration commit, reports a pre-registered, decisive pass at both rates (Table 1). Every BURST run had a hit fraction of 0.000 over the evaluation window, an uncached prefill rate of 1.009–1.029 times the all-miss rate and a mean backlog of 200–332 sessions. One (1.35 λ_{ref}, seed 1) is stuck in the same way, but its backlog was flat over the window (207 sessions at its start, 206 at its end; slope 0.0036 sessions/s, below the 0.0125 threshold), so the rule calls it indeterminate. Every NOBURST and CAPPED run is good (hit fractions 0.921–0.934, backlogs of 8–12 sessions, most asleep in tool calls); in the window CAPPED cannot be told apart from NOBURST, because the gate had drained the burst. Even pooled over the two rates the counts are small (Wilson 95% intervals: 6 of 6, [0.61, 1.00]; 5 of 6, [0.44, 0.97]); over the six seed-and-rate pairs, BURST's hit fraction is below NOBURST's in all six (one-sided sign test, $p = 1/64$).

In Fig. 1, every BURST run is at zero hits from the window starting at 1300 s to the end. Its backlog first fell, to 135–172 sessions 300–1300 s after the burst, as short burst sessions finished, then rose; over the last 300 s it was 206–369 sessions, still growing in five runs and flat in the sixth. The trapped engine still completed 0.081–0.097 sessions/s, above λ_{miss} because young sessions are cheap but below the arrival rate; only the uncached prefill rate sits at the all-miss level, which is why the registered rule uses tokens. The CAPPED gate's queue emptied 1547–1904 s after the burst at 1.35 λ_{ref} and 2021–2693 s at 1.65 λ_{ref}, between 2% shorter and 43% longer than the fluid estimate of Proposition 2 (1571 s and 1889 s) and well before the evaluation window. Outside the registered criteria, the CAPPED arm's hit fraction dips while the gate drains the burst: its lowest 100 s window is 0.48–0.77 in four of six runs, against 0.82–0.89 for NOBURST, which plausibly slowed the drain. So at loads that the engine carried from empty for the full 8700 s run (and for 3000 s in E0b), 200 sessions arriving within a minute, an order of magnitude more than the 8–12 present in the good state, moved it into an all-miss state that it did not leave within 7500 s, and a cap at the measured peak of $\mu(n)$ prevented that.

Table 2: Blind predictions (calibrated simulator, mixed batching) against outcomes; rates in sessions/s. B1 is not blind in time and is close to the calibration data (§4.4, Appendix C). B3 used a per-arm pass rule (each arm in 2 of 3 seeds); the registered rule asks for all three arms in the same seed. B4 is the registered seed 0 on the grid n = 48, 64, 96 (supplementary 5-seed means: peak 0.514, $\mu(96) = 0.085$, $\mu(300) = 0.108$).

	predicted	real
B1	λ_{miss} 0.0722 (turns)	0.0715 (+0.9%)
	11,220 tokens/s	11,243 (−0.2%)
	λ_{miss} 0.0694 (sessions)	0.0717 (−3.2%)
B2	all-stable edge 0.14 (3 seeds), 0.15 (20)	in [0.143, 0.179)
	at 0.18: 2/3 stable (15/20)	0.179 unstable
B3	PASS at all rates 0.09–0.16	PASS at 0.0966, 0.1180
	trapped token ratio 0.999–1.065	1.009–1.029
	trapped backlog 171–523 requests	194–325 requests
	good-arm hit 0.929–0.937	0.921–0.934
	trapped 3/3 at 0.09 and 0.10; 20 seeds: 14/20, 18/20	trapped 2/3 at 0.0966
B4	peak n = 64, μ 0.523 (900k)	n = 64, 0.479 (−8%)
	0.5-hit crossing ≈80	≈93
	$\mu(96) = 0.079$, $\mu(300) = 0.109$	0.123, 0.101

5.4 Blind predictions against outcomes

B1 matched within 3.2% (Table 2), which mostly checks the simulator's bookkeeping (§4.4). B2's edge, 0.14–0.15 sessions/s, is consistent with the real edge in

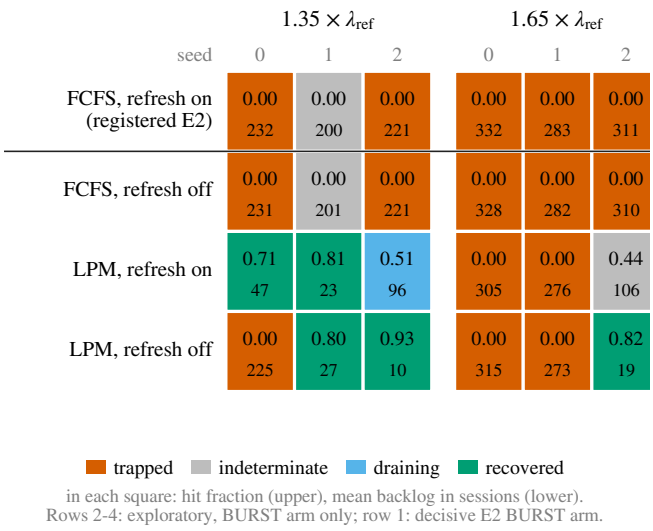


Figure 4: Exploratory policy grid, BURST arm only, 300k pool. Rows: FCFS with the waiting-prefix refresh (the registered runs), FCFS without, LPM with, LPM without. Columns: 1.35 and 1.65 λ_{ref} by seed. Colour: state over the evaluation window; numbers: hit fraction and mean sessions in the system.

[0.143, 0.179); near 0.179 the model keeps 2 of 3 runs stable (15 of 20 with more seeds), so one unstable run there is no contradiction. B3 predicted a pass at every tested rate in both batching modes. The observed trapped token rates (1.009–1.029) lie inside the predicted 0.999–1.065, and the observed good-arm hit fractions (0.921–0.934) are close to, and at the low end partly below, the predicted 0.929–0.937. B3 also flagged the low end as fragile, because the slope margin $0.5(\lambda - \lambda_{\text{miss}})$ is small there: with its registered three seeds it trapped 3 of 3 at both 0.09 and 0.10, but with 20 seeds 14 and 18, so 10–30% of runs there are stuck without meeting the slope test; one indeterminate real run in three at 0.0966 is consistent with that. (Backlogs compare only roughly: the simulator counts waiting requests, the engine runs outstanding ones.) B4 got the peak's position, within its coarse grid, and the outward shift of the cliff; the predicted peak was 9% above the real one and the cliff slightly early (at $n = 96$ the engine still hit 42% of turns), partly because the real pool is 6% larger than the simulated 900k. Two qualifications: the verifier's strict check that every B3 pass lies inside the B2 all-stable range failed (B3 asks for 2 of 3 seeds over a longer run), and B2 is blind by access control and commit order, not by event order: E0b had finished on the cluster before the predictions were committed, and was pulled 13 minutes after they were pushed.

5.5 Exploratory policy grid

Three other engine cells ran the BURST arm at the registered rates and seeds (Fig. 4) under exploratory registrations derived from E2's, with no control arms. The refresh does not matter for FCFS: all six runs without it have the registered runs' states, hit fraction 0.000 and backlogs within a few percent (200–231 sessions at 1.35 λ_{ref}, 282–328 at 1.65), as the source predicts: the refresh moves waiting prefixes up the LRU order but does not pin them.

LPM partly helps. From the 200-session burst, at 1.35 λ_{ref} it recovered in 4 of 6 runs and was draining in a fifth (with the refresh: recovered at hit fractions 0.71 and 0.81, draining at 0.51; without: recovered at 0.80 and 0.93, trapped once). At 1.65 λ_{ref} it was trapped in 4 of 6, indeterminate in one (hit 0.44) and recovered in one (0.82). This is a finite-burst result for the BURST arm alone, not an estimate of LPM's congested service rate μ_∞; with mixed outcomes in every cell, three seeds do not bracket a threshold. Under deep congestion stock LPM may behave like FCFS: Stage C's LPM run at $n = 300$ had zero hits with a queue above 128, and round 5 of our earlier simulator, which modelled the 128-request fallback, found no LPM advantage in congestion. Our simulator's pure LPM, without the fallback, has a congested service rate of about 1.17 λ_{miss} and recovered from 300 cold sessions only up to about 1.1 λ_{miss} (§7); these are deep-congestion quantities and do not compare directly with recovery from a finite burst. We do not claim that LPM fails, and we have not isolated why it recovered; candidates are partial prefix hits, which the simulator omits, and the 128-request fallback switching off once the queue shrinks. Seed 0 at 1.35 λ_{ref} recovered with the refresh and was trapped without it, so single runs near LPM's threshold are sensitive to small differences.

5.6 Running and planned

With the native pool $\mu^*/\lambda_{\text{miss}}$ is 6.7, so its window may be much wider; its from-empty edge is [pending: running: native E0b, jobs 17100938–39] and its kill test [pending: planned, not yet submitted: native blind prediction, registration and E2]. vLLM 0.32.0 passed its smoke test on the same GPU type (token ids and cached-token counts returned, flush verified); its all-miss capacity is [pending: running: vLLM E0a, job 17100940] and its BURST runs [pending: planned, not yet submitted: six vLLM BURST runs under an exploratory registration].

6 Robustness in the simulator

The real test fixes one workload, one burst size and one horizon; we varied each in the calibrated simulator (mixed batching). Each workload gets its own window $[1, hi]$ in multiples of its own λ_{miss} (hi: the largest multiple at which all 10 from-empty runs stay stable) and is tested one third and two thirds of the way across it; a point passes if each arm meets its criterion in at least 7 of 10 seeds, counted separately per arm (the registered real-engine rule instead counts seeds in which all three arms succeed).

Tool-time laws. With exponential tool times of mean 1, 2 and 5 s, a lognormal law (mean 5 s, coefficient of variation 2)

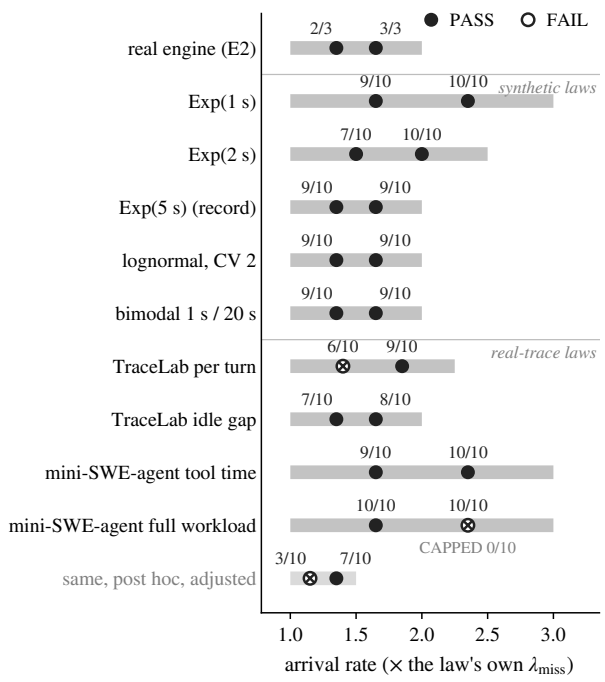


Figure 5: The paired test across tool-time laws (simulator). Gray bar: from-empty window $[1, hi]$ in multiples of each law's λ_{miss}. Markers: test points, filled = pass, crossed = fail (each arm counted separately, in ≥ 7 of 10 seeds); number = BURST runs trapped of 10. Top row: the real engine (3 seeds; registered rule, all three arms in at least 2 of the same seeds). Bottom row: labelled post-hoc rerun of the mini-SWE-agent full workload.

and a bimodal law (80% at 1 s, 20% at 20 s), λ_{miss} stays within 0.0718–0.0730 sessions/s: with the token profile fixed, it does not depend on the tool-time law. Shorter tool times raise the peak ($\mu^*/\lambda_{\text{miss}}$ is 5.25, 4.57 and 2.97 for means of 1, 2 and 5 s) and widen the window (hi = 3.0, 2.5, 2.0); heavy tails at the same mean leave the edge at 2.0. The test passes at all 10 points (Fig. 5): NOBURST and CAPPED succeed in 10 of 10 seeds everywhere, and BURST is trapped in 7 to 10 of 10.

Tool times from real agent traces. We replaced the tool-time law by empirical tables from SyFI TraceLab, real sessions of two commercial coding agents from 52 pseudonymous users (release v0.0.2, a later snapshot than the one the paper describes) [65] (per-turn mean 8.51 s with calls capped at 600 s, median 0.237 s, coefficient of variation 4.65; and an inter-request idle gap of mean 14.52 s as an upper bound), and from mini-SWE-agent v2.0.0 [53, 59] on the Verified subset of SWE-bench [28, 42] (mean 1.40 s, median 0.104 s, coefficient of variation 4.29). λ_{miss} moved by -0.8% to $+1.3\%$. The peak moved to $n = 32$ (TraceLab per turn) and 40 (idle gap) and stayed at 24 with mini-SWE-agent's tool times (cap 20); at every point of these three laws a cap at the new peak recovered in 10 of 10 seeds. The exponential control reproduced the record workload exactly and passed at both points. Of the six points of the three laws, five passed; the failure, TraceLab per turn at 1.40 λ_{miss}, is the slope rule: 6 of 10 BURST runs met it, all 10 were stuck below a hit fraction of 0.2, and on 20 fresh seeds (post hoc) 17 met it. A secondary workload with mini-SWE-agent's own turn and token counts passed at 1.65 λ_{miss} and failed at 2.35, where the cap of 12 had not drained the burst by the evaluation window (CAPPED recovered in 0 of 10; NOBURST good in 8 of 10). Its sessions outlive the declared measurement windows (λ_{miss} is 0.0120 sessions/s as declared, 0.0141 with a 20 times longer window); a labelled post-hoc rerun with longer horizons gives a narrower window, [1, 1.5], a pass at 1.35, and at 1.15 a failure of the slope rule (3 of 10 trapped, all 10 stuck below a hit fraction of 0.2).

Burst size and horizon. At 1.35 and 1.65 λ_{miss}, a burst of 25 sessions never traps (0 of 10), 50 is a coin flip (5 and 6 of 10 trapped), and from 100 up every run is stuck below a hit fraction of 0.2. In runs extended to 30,000 s after the burst, no trapped BURST run ever returned to a 100 s window above a hit fraction of 0.5, all 20 ended with 468 to 1065 waiting requests, and no NOBURST run ever exceeded 50.

The slope rule is the fragile part. Over the 100 BURST runs of the tool-time test the registered rule labels 90 trapped and 10 indeterminate, while all 100 are stuck at zero hits with an all-miss token rate. At 1.35 λ_{miss} with 20 fresh seeds, 3 are indeterminate (Wilson interval [0.05, 0.36]; median slope $0.77(\lambda - \lambda_{\text{miss}})$ against a threshold of 0.5), so the real engine's one in three there is not surprising. Over 31,200 s runs the rule labels only 10 to 12 of 20 stuck BURST runs trapped, depending on the window. Post hoc, a rule on the hit fraction alone would have called all six real BURST runs trapped; we report the registered rule.

The cap must follow the measured curve. The right cap, 20 for the workload of record at 300k, moves to 32 or 40 with the TraceLab tool-time laws (it stays at 20 with mini-SWE-agent's) and to 64–80 with the native pool, and with mini-SWE-agent's long sessions a cap of 12 drained the burst too slowly at 2.35 λ_{miss}; a cap above the new peak leaves $(\mu(N), \mu^*)$ open (Proposition 2).

7 Published mechanisms in simulation

A mechanism raises the recovery threshold to the extent that it raises μ_∞. We compare mechanisms by $\mu(300)/\lambda_{\text{miss}}$ as a proxy for μ_∞ (Fig. 6) in the earlier, uncalibrated simulator, with each rule checked against the released code where it exists (PEEK; the Continuum preview, which omits the paper's estimator) or otherwise against the paper (CONCUR), and corrected where it differed. These are results about the rules in a workload with private prefixes and exponential 5 s tools, not benchmarks of the named systems.

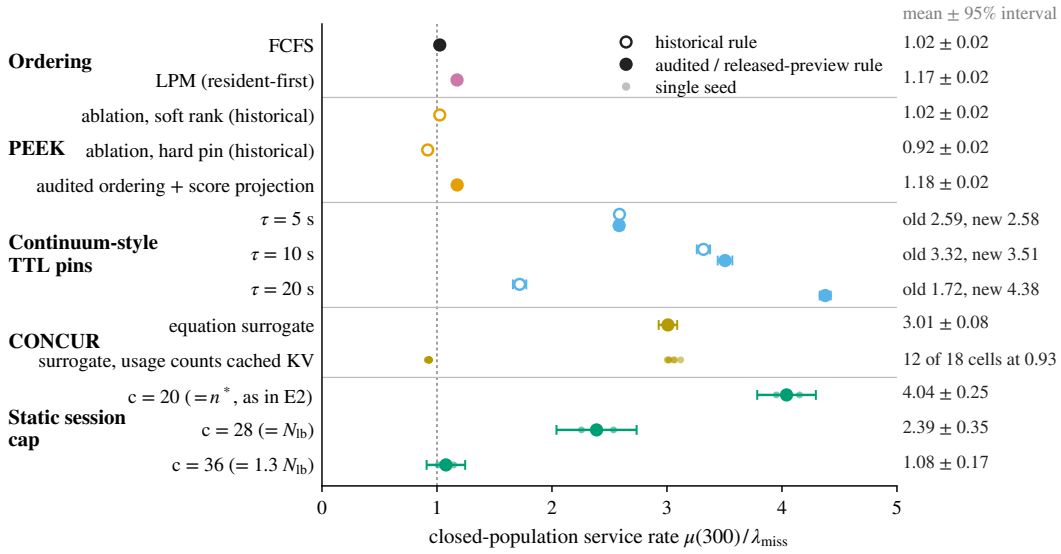


Figure 6: Congested service rate $\mu(300)/\lambda_{\text{miss}}$ per mechanism in the earlier, uncalibrated simulator: mean over 10 seeds with 95% t intervals, in units of that simulator's λ_{miss} (0.0474 sessions/s; not comparable with Figs. 2–3). Hollow: historical rows (PEEK as FCFS queued-session protection; Continuum pins released only on an idle GPU). Filled: projections of the released code. CONCUR: a surrogate of its control equation; small dots: 18 cells in which finished sessions' KV counts as usage. Static caps $c = 20, 28, 36$: $\mu(c)$ of the closed FCFS curve (3 seeds).

Orderings. FCFS gives 1.024 ± 0.020, as Proposition 1 predicts. LPM gives 1.174 ± 0.023 and recovers from 300 cold sessions over 150,000 s up to 1.1 λ_{miss}, not at 1.25. PEEK [58] protects prefixes needed by queued requests and adds a cache-aware scheduler. Our earlier rows implemented only the protection under FCFS order: as a soft ranking it is identical to FCFS seed for seed, and as a hard pin it gives 0.919. The released scheduler serves warm requests first; projected onto single-session prefixes with its eviction score, it gives 1.176 ± 0.021, equal to LPM within 0.001 ± 0.014, and recovers from none of 5 bursts at 2 λ_{miss}. Here PEEK is an LPM-class ordering, not FCFS; we did not evaluate it on shared-prefix traffic, for which it is designed.

TTL pins. Continuum pins a session's KV for a time τ after each turn so that it survives the tool call [32]. Under its released preview's rule (when an allocation fails and nothing runs, release the pin that expires last), $\tau = 5$, 10 and 20 s give 2.584 ± 0.035, 3.505 ± 0.064 and 4.376 ± 0.048; at $\tau = 20$ s the system recovers from a cold start up to 4.0 λ_{miss} and from 5 of 5 bursts. Our earlier rule waited for expiry while the GPU idled and gave 1.719 at $\tau = 20$ s; releasing pins whenever work waits, a rule the preview lacks, gives about 1.16 and recovers nowhere. Our reading is that pins help because pinned KV cannot be reclaimed by waiting misses, which bounds the set of sessions competing for memory. The rows lack Continuum's tool-conditional eligibility, TTL estimator and preemption.

Adaptive caps. CONCUR adds 2 to a window on active agents when KV usage is below 0.2 and halves it when usage exceeds 0.5 and the hit rate is below 0.2 [10]; we found no released code. Our session-gate surrogate of this equation gives 3.009 ± 0.081 and recovers from 5 of 5 bursts. It depends on the usage signal: if finished sessions' KV, still cached and evictable, counts as usage, 12 of 18 cells stall at the minimum window of 4 at 0.915–0.935 λ_{miss}; of five exploratory sensitivities, one (a 5 s control interval instead of 10 s) raised the rate by 0.184 ± 0.167, an unadjusted pointwise interval. Pause and resume of admitted agents is not modelled. So 3.0 is the surrogate's rate under a live-sessions-only usage signal, not deployed CONCUR's threshold.

Static caps. By Proposition 2 a cap c holds a large population at $\mu(c)$: 4.04 ± 0.25 at $c = 20$, 2.39 ± 0.35 at $c = 28 = N_{\text{lb}}$ and 1.08 ± 0.17 at $c = 36$ (3 seeds), against a peak of 4.07 at $n = 22$. The cliff makes the choice fragile: a cap 30% above N_{lb} is back near λ_{miss}.

Admission by memory demand. ThunderAgent pauses and restores whole programs when their total memory demand exceeds the KV capacity, between high and low watermarks [29], and NVIDIA Dynamo ships a port that pauses programs at tool boundaries [17]. This is the deployed form of a session bound, sized by memory demand rather than count; we have not evaluated it. On our reading, a gate that admits until mean demand reaches the pool would, by Proposition 2, hold the engine near N_{lb}, where in this simulator a static cap gave 2.39 against 4.04 at $c = 20$; its benefit therefore depends on its

threshold.

Why the ranking comes out this way. FCFS-type orderings cannot raise μ_∞ above λ_{miss}; resident-first orderings raise it only through tool calls shorter than the congested lifetime of idle KV; in this simulator, caps and pins, which bound the sessions competing for the pool (for pins, on our reading), raise it toward μ at the bound. Stock LPM did better on the real engine than here (§5.5), so the gap between orderings and caps may be smaller in practice.

8 A diagnostic, and what it implies

Choosing the number of admitted jobs from a measured throughput curve is the classical selection of a multiprogramming level [16, 22, 50], and stack-distance estimators size the KV working set for a target hit rate [33, 51], as does Yao et al.'s mean-field estimator for exogenous turn gaps [60]. What the window adds is that the from-empty edge, not only μ^*, bounds it; that a cap protects against history, not only throughput; and that the right cap moves with the tool-time law and the pool.

The window of Eq. (2) comes from three measurements that an operator can make before deployment, on the engine, model, pool and workload that will be served. (1) *All-miss capacity*: run a saturated closed population with prefix caching disabled and measure the uncached prefill token rate, which is stable across seeds, and convert it to sessions by the mean work per session or use turns per second over $E[T]$; completed sessions over a short window are noisy. By Proposition 1 this is also FCFS's recovery threshold when the congested engine stays busy, and an upper bound on it otherwise. (2) *Closed-population curve*: measure $\mu(n)$ on a grid that crosses the cliff, and do not read μ_∞ off the largest n of a short run with a synchronized start; here $\mu(300)$ was 0.108 sessions/s, 51% above λ_{miss}. (3) *From-empty edge*: run open arrivals from an empty engine at increasing rates over the horizon of interest.

If $\mu^* > \lambda_{\text{miss}}$, then under FCFS-type ordering every operating point in $(\lambda_{\text{miss}}, \min(\mu^*, \lambda_e))$ is one sufficiently large burst away from the congested state unless admission is bounded; a steady-state load test there passes without testing recovery. Here 200 sessions, about 20 times the 8–12 resident, trapped every run; in the simulator 25 never trapped and 50 trapped about half the time. Without a cap, Proposition 1 and the fluid model predict that recovering requires cutting the arrival rate below λ_{miss}, which at 300k tokens was 31% of peak throughput; the earlier simulator agrees (over 150,000 s, FCFS recovered from congestion at 0.9 λ_{miss} and not at 1.0), but no real run lowered the rate after a trap. Stock LPM, which Proposition 1 does not cover, recovered from the same burst at 1.35 λ_{miss} in 4 of 6 grid runs (§5.5). Proposition 2 suggests a session cap at the smallest n within a few percent of the peak (20 here), checked with a burst test.

The cap must be measured again when the workload or configuration changes: the peak moved from 20 to 32 or 40 sessions with the TraceLab tool-time laws in the simulator (it stayed at 20–24 with mini-SWE-agent's), and to 64–80 on the real engine with the 955,757-token pool. A cap sized for one setting can sit on the collapsed side of the curve in another.

9 Related work

Thrashing, caching and metastability. The thrashing collapse and the working-set remedy are Denning's [14, 15], and the decomposition analysis of thrashing instability and the choice of a multiprogramming level are classical (§2); LRU's failure on loops is classical [11, 52], as is its weakness on weak-locality patterns [27]. Metastable failures were named by Bronson et al. [7], surveyed by Huang et al. [25] and modelled as Markov chains by Alvaro et al. [2]; Gibbens, Hunt and Kelly's networks with random alternative routing, stabilized by trunk reservation, are the closest classical analogue of a cap removing a congested regime [21].

Queueing theory of LLM serving. Ao et al. give fluid-guided scheduling with threshold admission [3] and show that in a decode-only model, which abstracts away prefix reuse, the eviction-free fixed point can be unstable, with rate-limited admission and length mixing as remedies; its benchmark GPU experiments disable prefix caching [4]. Dai et al. prove throughput optimality of work-conserving policies without cache state, and give a cyclic-routing example where work-conserving scheduling fails [13]; Nie et al. give stability conditions under KV memory constraints for single requests [41]; Dong and Cao study flow-controlled admission [18]. Yao et al. derive a mean-field LRU hit ratio for multi-turn conversations with exogenous gaps between turns [60]; the feedback from load to gap length to hit rate is the loop studied here.

Prefix and KV mechanisms for agents. PEEK protects prefixes needed by queued requests and adds a cache-aware scheduler [58]; its "scheduling-induced thrashing" is LPM breaking ties by arrival order, not the 128-request fallback. Continuum pins KV across tool calls with a TTL and reports mean tool times of 925 ms on SWE-Bench (standard deviation 3550 ms) and 1923 ms on BFCL [32, 46]. CONCUR controls the number of active agents with AIMD and reports SGLang's hit rate falling from 77.7% at batch 32 to 35.4% at 40 [10]. ThunderAgent pauses programs whose memory demand exceeds the KV capacity and reports collapse without it under fixed concurrency [29]. CacheWise evaluates coding agents with sessions started together [54]; XPerf, a benchmark, sees hit rates fall toward saturation on rate up-sweeps [55]. Other systems manage idle KV across tool calls or workflow steps by offloading, prefetching or retention [1, 6, 36, 43, 57, 63], schedule agent programs [34, 38], cache conversation state hierarchically [20, 47, 62] (Mooncake's scheduler also rejects requests early under overload), or make locality-aware scheduling fair [9]. Eviction-order work [37, 44, 56] changes which idle prefix is evicted; RR-Evict assumes that the active trajectories fit in the cache, and Liu et al. replay traces without queueing feedback. Close neighbours: Metronome calls a collapse in real-time speech-model serving metastable, but there

each session's KV stays pinned for the whole conversation until the pinned KV exhausts the pool, a different mechanism from eviction and recomputation of reusable prefixes [39]; MISA-T derives an adaptive admission cap for new sessions from observed KV demand and overload pressure [24]; and, known to us only from their abstracts, AgentServeSim simulates agent serving with KV retained across tool gaps [49] and ServeTwin simulates multi-turn agent workloads while tracking KV-cache evolution [45].

Position. Hit-rate collapse with too many active agents is published [10, 29, 54, 55], as is thrashing caused by scheduling order [58], and session-aware admission is deployed [17, 29]; we claim neither. To our knowledge no prior work on LLM serving shows the open-system, history-dependent version (a good state that persists from empty, a burst that leaves the system congested, and a cap that prevents it) against a no-burst control with a pre-registered rule on a real engine; in general distributed systems, the cache-loss version is Bronson et al.'s look-aside-cache case [7]. Extending the models of Ao et al. [4] or Yao et al. [60] so that inter-turn gaps depend on load would cover our setting; neither does so.

10 Limitations and threats to validity

One configuration, synthetic sessions. All registered evidence is for SGLang 0.5.22, Qwen2.5-7B-Instruct, one A100 and a 300k-token pool; the native kill test and vLLM are pending; larger models, parallelism, other GPUs and offloading are untested. Sessions are synthetic (random tokens, fixed tool outputs and output lengths, geometric turn counts, exponential 5 s tools) and share no prefixes, which removes the case PEEK's scheduler targets. Production agent traces report a hit rate of about 55% across turn boundaries and user idle gaps of minutes [35], while 93% of our turns hit in the good state (not the same metric). The simulator suggests that real tool-time laws keep the trap but move the window and the cap; the engine has not tested this.

Few runs, finite horizons, uncontrolled factors. Stage C and E0b have one run per point, so the peak's location (20 against 24) is within noise and single runs near the cliff land on either side of it; E2 and the grid have three seeds. "Does not leave" means within 7500 s on the engine and 30,000 s in the simulator; E0b's 3000 s horizon cannot see slow escapes, so the true edge may be below 2.0 λ_{ref}. The slope test loses margin near λ_{miss}. In Stage C the cliff ($n = 32$ to 48) coincides with a change of node and job; the native curve and E2, which ran on three nodes, argue against a node effect. The SGLang server seed was not pinned, and the load client runs on the server's node.

Model and simulators. Eq. (1) is used only for fixed points, recovery and a drain estimate, which the engine took up to 43% longer to complete; Proposition 1 is asymptotic, gives equality only when the congested engine stays busy, and assumes congested misses cost what cache-disabled ones do; that recovery without a cap needs a rate below λ_{miss} is untested on the engine; LPM's behaviour is a conjecture; the capped system's stochastic stability is not proved. The calibrated simulator failed its level test at $n = 48$, reserves whole turns, omits partial prefixes and the refresh, and understated LPM. The mechanism comparison uses an older simulator, and its PEEK, Continuum and CONCUR rows are projections or surrogates.

11 Conclusion

Prefix caching turns multi-turn agent serving into a thrashing workload: the cache that makes follow-up turns cheap belongs to sessions idle in tool calls, and admission reclaims it first. In our SGLang configuration the engine stayed in the good state from empty up to twice the all-miss capacity over 3000 s, and at 1.35 and 1.65 times that capacity the good state was metastable: in a pre-registered test a burst moved the engine into an all-miss state that it did not leave within 7500 s, the no-burst and capped arms stayed good, and a timing-calibrated simulator had predicted this. Operators should measure the closed-population curve for the workload they serve, cap admitted sessions near its peak, and re-measure when the workload changes.

References

[1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiying Zhang. InferCept: Efficient intercept support for augmented large language model inference. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 81–95, 2024.

[2] Peter Alvaro, Rebecca Isaacs, Rupak Majumdar, Kiran-Kumar Muniswamy-Reddy, Mahmoud Salamati, and Sadegh Soudjani. Formal analysis of metastable failures in software systems. arXiv:2510.03551, 2025.

[3] Ruicheng Ao, Gan Luo, David Simchi-Levi, and Xinshang Wang. Optimizing LLM inference: Fluid-guided online scheduling with memory constraints. arXiv:2504.11320 (v4, 13 June 2026), 2025.

[4] Ruicheng Ao, Jing Dong, Gan Luo, and David Simchi-Levi. Service-induced congestion in memory-constrained LLM serving. arXiv:2606.15555 (v3, 20 October 2026), 2026.

[5] L. A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2): 78–101, 1966.

[6] Zhuohang Bian, Feiyang Wu, Zhuoran Li, Teng Ma, and Youwei Zhuo. TokenCake: A KV-cache-centric serving framework for LLM-based multi-agent applications. arXiv:2510.18586; to appear at EuroSys '27, 2025.

[7] Nathan Bronson, Abutalib Aghayev, Aleksey Charapko, and Timothy Zhu. Metastable failures in distributed systems. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '21)*, Ann Arbor, MI, 2021. doi: 10.1145/3458336.3465286.

[8] Shelby L. Brumelle. On the relation between customer and time averages in queues. *Journal of Applied Probability*, 8(3):508–520, 1971. doi: 10.2307/3212174.

[9] Shiyi Cao, Yichuan Wang, Ziming Mao, Pin-Lun Hsu, Liangsheng Yin, Tian Xia, Dacheng Li, Shu Liu, Yineng Zhang, Yang Zhou, Ying Sheng, Joseph Gonzalez, and Ion Stoica. Locality-aware fair scheduling in LLM serving. arXiv:2501.14312, 2025.

[10] Qiaoling Chen, Zhisheng Ye, Tian Tang, Peng Sun, Boyu Tian, Guoteng Wang, Shenggui Li, Yonggang Wen, Zhenhua Han, and Tianwei Zhang. CONCUR: High-throughput agentic batch inference of LLM via congestion-based concurrency control. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 17881–17891, 2026. arXiv:2601.22705.

[11] Hong-Tai Chou and David J. DeWitt. An evaluation of buffer management strategies for relational database systems. In *Proceedings of VLDB 85*, pages 127–141, Stockholm, 1985.

[12] P. J. Courtois. Decomposability, instabilities, and saturation in multiprogramming systems. *Communications of the ACM*, 18(7):371–377, 1975. doi: 10.1145/360881.360887.

[13] J. G. Dai, Tianze Deng, Yueying Li, and Tianyi Peng. Throughput-optimal scheduling algorithms for LLM inference and AI agents. arXiv:2504.07347 (v3, 18 May 2026), 2025.

[14] Peter J. Denning. Thrashing: Its causes and prevention. In *Proceedings of the AFIPS Fall Joint Computer Conference*, pages 915–922, 1968.

[15] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968. doi: 10.1145/363095.363141.

[16] Peter J. Denning, Kevin C. Kahn, Jacques Leroudier, Dominique Potier, and Rajan Suri. Optimal multiprogramming. *Acta Informatica*, 7(2):197–216, 1976. doi: 10.1007/BF00265771.

[17] Ishan Dhanani, Jamie Li, and Karen Chung. Session-aware agentic inference with NVIDIA Dynamo. PyTorch Blog, 8 October 2026; `https://pytorch.org/blog/session-aware-agentic-inference-with-nvidia-dynamo/`, 2026.

[18] Zhuolun Dong and Junyu Cao. Flow-controlled scheduling for LLM inference with provable stability guarantees. arXiv:2604.11001, 2026.

[19] Guy Fayolle, Erol Gelenbe, and Jacques Labetoulle. Stability and optimal control of the packet switching broadcast channel. *Journal of the ACM*, 24(3):375–386, 1977. doi: 10.1145/322017.322019.

[20] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *USENIX Annual Technical Conference (USENIX ATC 24)*, 2024. arXiv:2403.19708.

[21] R. J. Gibbens, P. J. Hunt, and F. P. Kelly. Bistability in communication networks. In G. R. Grimmett and D. J. A. Welsh, editors, *Disorder in Physical Systems*, pages 113–127. Oxford University Press, 1990.

[22] Hans-Ulrich Heiss and Roger Wagner. Adaptive load control in transaction processing systems. In *Proceedings of the 17th International Conference on Very Large Data Bases (VLDB '91)*, pages 47–54, Barcelona, 1991. Morgan Kaufmann.

[23] Daniel P. Heyman and Shaler Stidham, Jr. The relation between customer and time averages in queues. *Operations Research*, 28(4):983–994, 1980. doi: 10.1287/opre.28.4.983.

[24] Zetao Hong, Song Yuan, Yuanhao Ding, Yibo Zhu, Daxin Jiang, Zhibin Wang, and Chen Tian. Scheduling mixed RL rollouts beyond prefix locality. arXiv:2608.11152 (v1, 11 August 2026), 2026.

[25] Lexiang Huang, Matthew Magnusson, Abishek Bangalore Muralikrishna, Salman Estyak, Rebecca Isaacs, Abutalib Aghayev, Timothy Zhu, and Aleksey Charapko. Metastable failures in the wild. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 73–90, 2022.

[26] Van Jacobson. Congestion avoidance and control. In *Symposium Proceedings on Communications Architectures and Protocols (ACM SIGCOMM '88)*, pages 314–329, 1988. doi: 10.1145/52324.52356.

[27] Song Jiang and Xiaodong Zhang. LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance. *ACM SIGMETRICS Performance Evaluation Review*, 30(1):31–42, 2002. doi: 10.1145/511399.511340.

[28] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR 2024)*, 2024.

[29] Hao Kang, Ziyang Li, Weili Xu, Xinyu Yang, Yinfang Chen, Junxiong Wang, Beidi Chen, Tushar Krishna, Chenfeng Xu, and Simran Arora. ThunderAgent: A simple, fast and program-aware agentic inference system. arXiv:2602.13692 (v3, 30 June 2026), 2026.

[30] Leonard Kleinrock and Simon S. Lam. Packet switching in a multiaccess broadcast channel: Performance evaluation. *IEEE Transactions on Communications*, 23(4): 410–423, 1975. doi: 10.1109/TCOM.1975.1092814.

[31] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP 2023)*, 2023.

[32] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. arXiv:2511.02230 (v7, 8 September 2026), 2025.

[33] Luchang Li, Shuaishuai Wang, Zhao Ruan, Dongfang Li, and Bozhao Gong. The KV cache working set: Online capacity planning for LLM inference systems. arXiv:2609.27746 (v1, 23 September 2026), 2026.

[34] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of LLM-based applications with semantic variable. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024. arXiv:2405.19888.

[35] Banruo Liu, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, and Esha Choukse. Agentic coding in the wild: Characterizing GitHub Copilot traces at production scale. arXiv:2608.00101 (v1, 30 July 2026), 2026.

[36] Yipeng Liu, Yingqiang Zhang, Feifei Li, and Huanchen Zhang. Ask the tool, don't guess: Agent tool calls hold their progress, and the serving system should read it. arXiv:2609.18849, 2026.

[37] Yiyu Liu, Minlan Yu, and Juncheng Yang. When fancy eviction fails: Rethinking cache replacement for LLM prefix reuse. arXiv:2609.28870 (v2, 1 October 2026), 2026.

[38] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. Agentix: An efficient serving engine for LLM agents as general programs. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*, 2026. arXiv:2502.13965 (as Autellix).

[39] Jiaying Meng and Bojie Li. Metronome: Bound the cache, keep the beat for real-time interaction model serving. arXiv:2607.02640 (v1, 2 July 2026), 2026.

[40] John Nagle. Congestion control in IP/TCP internetworks. RFC 896, 6 January 1984; `https://www.rfc-editor.org/rfc/rfc896`, 1984.

[41] Chengyi Nie, Nian Si, and Zijie Zhou. A queueing-theoretic framework for stability analysis of LLM inference with KV cache memory constraints. arXiv:2605.04595; to appear at ICML 2026, 2026.

[42] OpenAI. Introducing SWE-bench Verified. OpenAI blog post, 13 August 2024; `https://openai.com/index/introducing-swe-bench-verified/`, 2024.

[43] Zaifeng Pan, Ajjkumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. In *Advances in Neural Information Processing Systems (NeurIPS 2025)*, 2025. arXiv:2507.07400.

[44] Zaifeng Pan, Chris Wu, Zhengding Hu, Xinwei Qiang, Zhongkai Yu, and Yufei Ding. RR-Evict: Fine-grained prefix cache eviction beyond LRU for agentic LLM serving. arXiv:2609.32278 (v2, 2 October 2026; under review), 2026.

[45] Sungjoon Park, Changue Jung, Kyungno Joo, Mincheol Kang, Jaehyung Ahn, Sehwan Lee, and Sangjoon Kim. ServeTwin: A benchmark-validated simulator for distributed LLM architecture exploration. arXiv:2610.02732 (v1, 2 October 2026), 2026.

[46] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392, 2025.

[47] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, 2025. arXiv:2407.00079.

[48] Qwen Team. Qwen2.5 technical report. arXiv:2412.15115, 2024.

[49] Rakibul Hasan Rajib, Mengxin Zheng, and Qian Lou. AgentServeSim: Serving-system simulation and policy search for LLM agent programs. arXiv:2606.09613 (v3, 5 September 2026), 2026.

[50] Bianca Schroeder, Mor Harchol-Balter, Arun Iyengar, Erich Nahum, and Adam Wierman. How to determine a good multi-programming level for external scheduling. In *Proceedings of the 22nd International Conference on Data Engineering (ICDE '06)*, page 60, 2006. doi: 10.1109/ICDE.2006.78.

[51] Kunming Shao, Jierun Chen, Jiangnan Yu, Xiao-Hui Li, Chaofan Tao, Yanli Wang, Huanxin Lin, Kwang-Ting Cheng, Chi Ying Tsui, and Haoli Bai. EfficientAgent: What makes KV cache offloading work for concurrent agents? arXiv:2609.33762 (v1, 27 September 2026), 2026.

[52] Yannis Smaragdakis, Scott Kaplan, and Paul Wilson. EELRU: Simple and effective adaptive page replacement. In *Proceedings of the 1999 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, pages 122–133, 1999. doi: 10.1145/301453.301486.

[53] SWE-agent. mini-SWE-agent. GitHub repository, release v2.0.0 (11 February 2026); `https://github.com/SWE-agent/mini-swe-agent`, 2026.

[54] Shubham Tiwari, Tapan Chugh, Nash Rickert, Simon Peter, Ratul Mahajan, and Haiying Shen. CacheWise: Understanding workloads and optimizing KV-Cache management for efficiently serving LLM coding agents. arXiv:2606.16824, 2026.

[55] Michael Wang, Yikang Yue, Shaobo Li, Yirui Eric Zhou, Chen Wang, and Jian Huang. Benchmarking LLM serving systems for agentic AI workloads with XPerf. arXiv:2608.20370, 2026.

[56] Fangzhou Wu, Sandeep Silwal, and Qiuyi (Richard) Zhang. Randomization boosts KV caching, learning balances query load: A joint perspective. In *International Conference on Learning Representations (ICLR 2026)*, 2026. arXiv:2601.18999.

[57] Tian Xia, Hanchen Li, Zhifei Li, Xiaokun Chen, Hao Kang, Yifan Qiao, Yi Xu, and Ion Stoica. Idleness is relative: Exploiting tool-call idle windows for offloading in agentic systems with MORI. arXiv:2606.00866, 2026.

[58] Bing Xie, Zhipeng Wang, Masahiro Tanaka, and Zhen Zheng. PEEK: Predictive queue-informed KV cache management for LLM serving. arXiv:2607.02525 (v2, 12 September 2026; under review), 2026.

[59] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. arXiv:2405.15793 (v3, 11 November 2024), 2024.

[60] Heyuan Yao, Chutong Gao, Yuan Lyu, Izzy Grosof, and David Simchi-Levi. Multi-turn LLM conversations under the least-recently-used policy: Mean-field asymptotics and hit ratio approximation. arXiv:2609.02027, 2026.

[61] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022.

[62] Lingfan Yu, Jinkun Lin, and Jinyang Li. Stateful large language model serving with pensieve. In *Proceedings of the European Conference on Computer Systems (EuroSys '25)*, 2025. doi: 10.1145/3689031.3696086. arXiv:2312.05516.

[63] Rui Zhang, Chaeeun Kim, Shaoting Feng, Kuntai Du, Yuhan Liu, Yi Zhong, Cheng-Wei Ching, Junchen Jiang, and Liting Hu. Learning agent execution for KV-cache management in agentic serving. arXiv:2608.14624, 2026.

[64] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 62557–62583, 2024.

[65] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. TraceLab: Characterizing coding agent workloads for LLM serving. arXiv:2606.30560 (v2, 30 June 2026), 2026. Trace release v0.0.2, 24 July 2026: `https://github.com/uw-syfi/TraceLab`.

A Protocol details

Table 3: Real-engine stages (SLURM job IDs on Nova).

Stage	Jobs	What it measures
Stage 0	17060165	smoke test
Stage C	17062103–05	closed $\mu(n)$, 300k pool; FCFS $n \in \{16, 20, 24, 32, 48, 96, 300\}$, LPM $n \in \{48, 300\}$
E0a	17072634–37	λ_{miss}, cache off
E0b	17084750–55	from-empty stability
E2	17093387–404	registered test, 2 rates × 3 seeds × 3 arms
native C	17096856–57	$\mu(n)$, native pool, $n \in \{24, 48, 64, 80, 96, 128, 300\}$
native E0a	17096858	λ_{miss}, native pool
grid	17097086–103	BURST arm, 3 policy cells
vLLM	17097972–74	install, GPU check, smoke
native E0b	17100938–39	[pending: running]
vLLM E0a	17100940	[pending: running]

Table 4: Workload and engine of the real-engine runs.

First prompt L_0	2000 tokens
Tool output per turn Δ	550 tokens
Output per turn d	100 tokens, greedy, `ignore_eos`
Turns T	$\min(\text{Geometric}(1/15), 60)$, $E[T] = 14.761$
Tool time Z	exponential, mean 5 s
Prompt at turn k	$2000 + 650(k-1)$ tokens
Output feedback	server-returned token ids
Engine	SGLang 0.5.22, FlashInfer
Scheduling	FCFS, waiting-prefix refresh on
Batching	chunked prefill 8192, ≤ 256 running
KV pool	300,000 tokens; native 955,757
Context limit	49,152 tokens
Model	Qwen2.5-7B-Instruct, rev. a09a3545
GPU	A100 80GB PCIe, 81,920 MiB, driver 580.178.04

Stage 0. The smoke test (job 17060165) verified that every engine flag exists; that cached tokens are reported (a repeated 64-token prompt had 63 tokens cached); that the server returns output token ids; that contexts up to 40,350 tokens are accepted; that the KV pool is exactly 300,000 tokens; that client and server agree on prompt length (0 mismatches over 3191 requests); and that the flush empties the cache (canary 1023 cached tokens before, 0 after).

Stage C and E0a. Closed population; all sessions start at $t = 0$ and a finished session is replaced at once; 900 s warmup and an 1800 s measured window; one run per n (seed 0); flush and canary check between runs. E0a disables the radix cache and runs 32 sessions with seeds 0, 1 and 2, plus 64 sessions with seed 0 as a saturation check. Its per-run values over the window were (sessions/s, turns-based sessions/s, uncached tokens/s): seed 0, 0.0567, 0.0643, 11,076; seed 1, 0.0817, 0.0773, 11,342; seed 2, 0.0767, 0.0730, 11,312; 64 sessions, 0.0544, 0.0676, 11,431. The spread comes from in-flight composition: the mean turn index of the requests in the window was 15.9, 13.2 and 14.1 for seeds 0, 1 and 2.

E0b. Poisson session arrivals from an empty cache and queue at 1.25, 1.5, 1.75, 2.0, 2.5 and 3.0 λ_{ref}; one 3000 s run per rate; stable if the final queue is below 50 and the fitted rise of the queue over the last 1500 s is at most 10.

E2. One arm per 8700 s job, each on a fresh server; burst of 200 fresh sessions within 60 s at $t_b = 1200$ s for BURST and CAPPED; CAPPED gate $n_{\text{cap}} = 20$; evaluation window $[t_b + 4500, 8700]$ s. The registered rule (Table 5) was applied by the analysis tool frozen at the registration commit, with the same parameter overrides that the jobs themselves used.

Table 5: The registered decision rule.

Rate estimator	uncached prefill tokens/s
Reference rate	E0a all-miss, 11,243 tokens/s
Rate tolerance	±0.15
Slope test	slope > 0.5 $(\lambda - \lambda_{\text{miss}})$, 10 s bins
Minimum window	600 s
Recovered	hit > 0.8 and outstanding < 20, last 300 s
Hit metric	turns with cached/prompt > 0.5
Pass	≥ 2/3 of ≥ 3 seeds with all three arm conditions met in the seed

Backlog definitions. The figures and Table 1 report the backlog as sessions in the system (arrived minus ended), which includes sessions asleep in tool calls (8–12 in the good state). The registered RECOVERED test uses the outstanding backlog (requests in flight plus sessions held at the gate). The simulator's predictions count waiting requests plus the gate queue.

B The registration

The registration is `configs/agentload_prereg_2026-10-24.yaml`, status FILLED, committed in `d279b7b` (2026-10-24, 15:09 CDT) before the first E2 job, SHA-256 `2a6d680a9615bba962a5673d6db6b3e4ca5102bf778ca974cea46147f5cc0ac6`. Its fields:

- **Cell and inputs.** Engine cell `sglang-fcfs-r1` (SGLang FCFS, waiting-prefix refresh on); committed copies of the Stage C curve, the E0a calibration and the E0b bracket under `prereg/inputs/2026-10-23/`; protocol 8700 s, burst of 200 within 60 s at 1200 s.
- **Measured.** $\mu(n)$ in sessions/s: 0.1756 ($n = 16$), 0.2239 (20), 0.2333 (24), 0.1150 (32), 0.0489 (48), 0.0611 (96), 0.1078 (300). $\mu^* = 0.2333$; $n_{\text{cap}} = 20$ by the smallest-within-5% rule (argmax 24; flagged ambiguous). $\lambda_{\text{miss}} = 0.07167$ (sessions) and 0.07153 (turns-based, $= \lambda_{\text{ref}}$); all-miss rate 11,243 tokens/s; E0b largest all-stable multiple 2.0, first unstable 2.5, monotone; hardware A100 80GB PCIe.

- **Predicted boundaries.** Recovery threshold λ_{ref} (not $\mu(300)$, which a synchronized start biases upward). Window (0.0715, 0.1431) sessions/s = (1.0, 2.0) λ_{ref}, upper edge from E0b. CAPPED drain must fit within 80% of the 7200 s available after the burst (5760 s), which holds for λ up to 2.64 λ_{ref}; predicted drain 1571 s at 1.35 and 1889 s at 1.65.
- **Rates.** 1.35 and 1.65 λ_{ref} by the one-third/two-thirds rule, rounded to 0.05, inside the margin band (1.1, 1.8); seeds 0, 1, 2. The rate rule and the cap of 20 were committed to the runbook on 2026-10-22 (`288b801`, 13:13 CDT), before E0b was submitted; the registration's own text dates them 2026-10-23.
- **Predictions.** Inside the window: BURST trapped, NOBURST good through the run horizon, CAPPED recovered if the drain fits. Below λ_{ref}: BURST not trapped. Above the window: NOBURST may escape, so no such rate was registered.
- **Kill conditions.** NOBURST not good in at least one third of the seeds at a registered rate; BURST not trapped in two thirds of the seeds at every registered rate; CAPPED not recovering while BURST traps and the drain fits; a window that is empty or narrower than the noise.
- **Warnings recorded at filling.** $|\mu(300)/\lambda_{\text{miss}} - 1| = 0.507$ (a check only), and the ambiguous cap.

The verdict is in `results/agentload_analysis_burst2/20261025-144012/` inside the E2 data directory: pre-registered, decisive pass at 1.35 and 1.65 λ_{ref}. The three grid cells have their own derived registrations, `configs/agentload_prereg_2026-10-24_grid_<cell>.yaml` for the cells `sglang-fcfs-r0`, `sglang-lpm-r1` and `sglang-lpm-r0`, with status EXPLORATORY; the analysis never calls them decisive.

C Blind-prediction timeline

B3 (E2) and B4 (native pool) were committed before their experiments were submitted, so they are blind in time. B2 (E0b) was committed after E0b had finished on the cluster but before its results were pulled. The agents that made the predictions were instructed not to contact the cluster and not to open the E0a or E0b result directories or the main checkout; they could technically have done so, and we checked their session records afterwards and found no cluster command and no access to E0b before the tag was pushed. For B2 this instruction-level control, checked after the fact, is the only safeguard. The order of the commits, the tag and the pull was checked from the repository's reflog and the push record. A scan of every line added between the base commit and the predictions found no use of withheld data, but a text scan cannot prove that nobody read it. B1 (E0a) rests on the same instruction only: E0a was committed before the predictions. The independent verifier had read the project state file, which mentions E0a, so it is independent but not blind. Two conditions of the verification failed and are disclosed: the strict containment of B3 passes within the B2 all-stable range, and the platform-specific test failures of the full suite.

Table 6: Order of events (times CDT). Commits are in the project repository; job IDs on Nova.

When	Event
10-21	Stage 0 passes (17060165); Stage C submitted
10-22	Stage C results pulled; E0a jobs complete; simulator cost model fitted to Stage 0/C server logs (`4435c51`)
10-22 13:13	rate rule and cap of 20 committed to the runbook (`288b801`)
10-23 14:32	E0a results committed (`a76f5e5`)
10-23 ~14:40	E0b submitted (17084750–55); runs that afternoon
10-24 03:13	validation of the calibrated simulator committed (`813cfc1`)
10-24 03:23	prediction spec and code committed before any prediction run (`630ab9e`)
10-24 03:49	predictions B1–B4 committed (`64bc4e3`)
10-24 04:05	independent verification recorded (`877b936`)
10-24 14:52	tag `r6-predictions-frozen`; pushed 14:53
10-24 15:06	E0b results pulled (committed `cfbc702`)
10-24 15:09	registration committed (`d279b7b`)
10-24 15:13	E2 submitted (17093387–404)
10-24 15:38 to 10-25 ~02:40	E2 runs
10-25 02:28	native-pool Stage C and E0a start (17096856–58)
10-25 14:40	registered E2 verdict computed (data `49fbc7b`)

D Provenance

Every number in the paper is listed with its source file and commit or job in `paper/NUMBERS.md`. Commits dated before 2026-10-22 predate a history rewrite and are mapped to current hashes in `provenance/commit-map.tsv`.

Derived numbers. The CAPPED drain times, the BURST backlog after the burst and over the window, the E0b hit fractions and the interval estimates for the E2 counts are computed from committed run data by `figures/paper/derived_numbers.py` and written to `paper/derived/`.

Notes that support the text. The math check of the model is `notes/MATHCHECK_thrashing_curve.md`; the verification of Stage C (three-source hit check, eviction survival times, prompt continuity) is `notes/STAGEC_VERIFICATION.md`; the audit of the PEEK, Continuum and CONCUR rules against released code is `notes/BASELINES_AUDIT.md`; the audit of every citation, the scoop check and the reading of the SGLang and vLLM sources (read twice, through a page summariser, without verbatim code; the engine facts of §2 rest on it) are in `notes/lit/CITATION_AUDIT_2026-10-25.md`; the trace statistics are in `notes/lit/AGENT_TRACES.md`. Where the research log and a committed CSV differ in the last digit (three hit fractions and one backlog of the policy grid), the paper uses the CSV.

Table 7: Data behind each result (paths relative to the repository).

Result	Directory (commit)
Stage 0, C	results/nova/s0_smoke_17060165_pulled_20261021-152347/, results/nova/stageC_1706210x_pulled_20261022-125454/ (d0235a7)
E0a	results/nova/e0a_1707263x_pulled_20261023-142037/ (a76f5e5)
E0b	results/nova/e0b_1708475x_pulled_20261024-150639/ (cfbc702)
E2	results/nova/e2_170933xx_pulled_20261025-144558/ (49fbc7b)
native, grid, vLLM	results/nova/r7_native_grid_vllm_pulled_20261025-145126/ (fa265c2)
validation	results/r6_validate_stagec/20261024-025222/ (813cfc1)
B1–B4	results/r6_predict_report/20261024-034707/, notes/PREDICTIONS.md (64bc4e3)
robustness	results/r7_report/20261024-233554/ (c653498); results/r7_traces_sim/r7_report/20261025-012451/, results/r7_traces_sim_adj/r7_report/20261025-012453/ (b530e55)
mechanisms	results/prefix_meta_baselines_audit_summary/20261025-001850/ (dc76032); results/prefix_meta_r4_curve/20261021-050856/, results/prefix_meta_r4/20261021-051550/ (a1d485b)
figures	figures/paper/, paper/figs/README.md (e098e1c, c974070)
derived	paper/derived/, figures/paper/derived_numbers.py (38bf22b)